Introduction to Machine Learning Methods

Reinforcement Learning

Haris Gačanin
RWTH Aachen University

Outline

- Passive learning
- Active learning

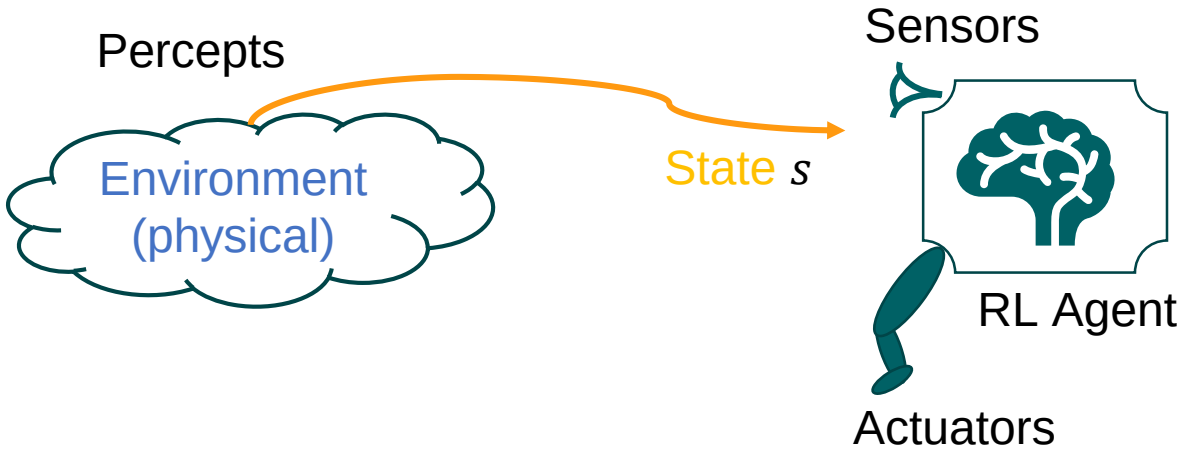
Reference book: Chapters 21

Reinforcement learning – concept

- Learn how to behave successfully to achieve a goal while interacting with an external environment
- Learn from experiences.
- Unlike classification there are no examples of correct answers: try to learn.

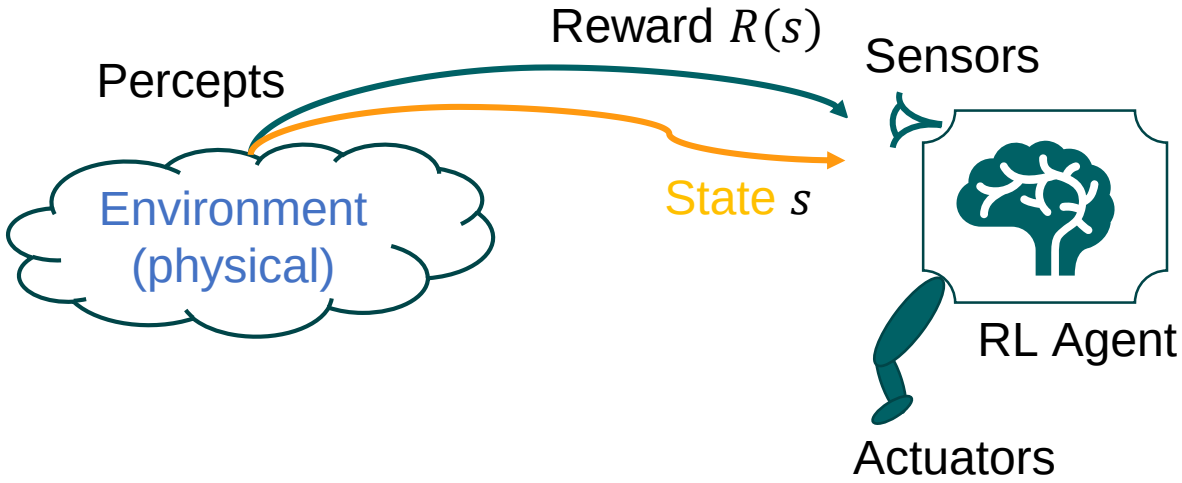
Reinforcement learning – concept

- Learn how to behave successfully to achieve a goal while interacting with an external environment
- Learn from experiences.
- Unlike classification there are no examples of correct answers: try to learn.



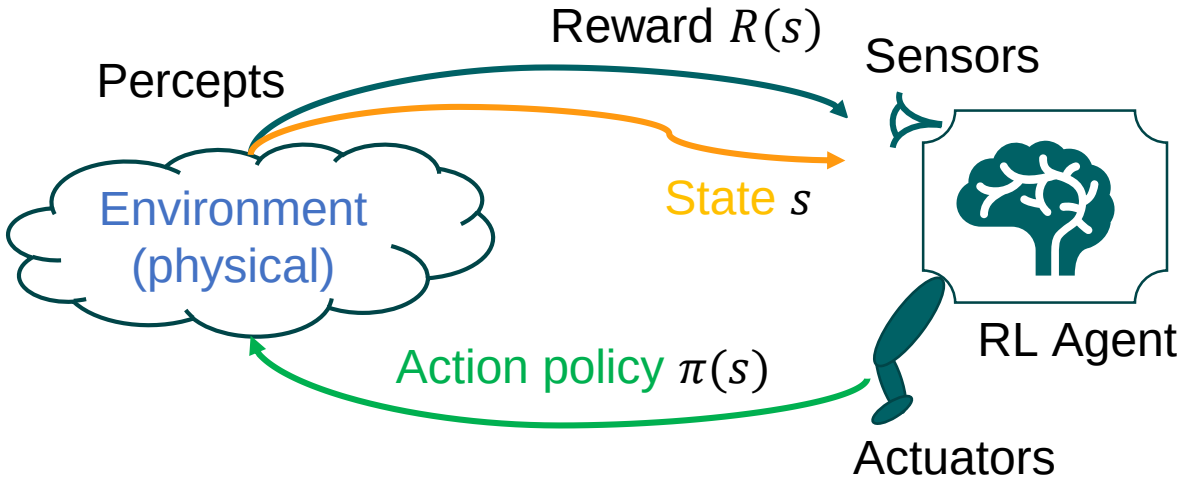
Reinforcement learning – concept

- Learn how to behave successfully to achieve a goal while interacting with an external environment
- Learn from experiences.
- Unlike classification there are no examples of correct answers: try to learn.



Reinforcement learning – concept

- Learn how to behave successfully to achieve a goal while interacting with an external environment
- Learn from experiences.
- Unlike classification there are no examples of correct answers: try to learn.



RL model

- Reinforcement:
  - Change the state and trigger rewards or punishments (i.e., negative rewards)
  - Each percept is enough to determine the state (the state is accessible)
  - ***Note*:** The agent can decompose the reward component from a percept.

RL model

- Reinforcement:
  - Change the state and trigger rewards or punishments (i.e., negative rewards)
  - Each percept is enough to determine the state (the state is accessible)
  - ***Note*:** The agent can decompose the reward component from a percept.
- Learns an action policy for acting:
  - Find an optimal policy $\pi(s)$ to map states to actions that maximizes the long-run measure of the reinforcement.
  - Maximize cumulative reward during an episode.

RL model

- Reinforcement:
  - Change the state and trigger rewards or punishments (i.e., negative rewards)
  - Each percept is enough to determine the state (the state is accessible)
  - ***Note*:** The agent can decompose the reward component from a percept.
- Learns an action policy for acting:
  - Find an optimal policy $\pi(s)$ to map states to actions that maximizes the long-run measure of the reinforcement.
  - Maximize cumulative reward during an episode.
- Exploration-Exploitation **trade-off**
  - Exploration: Get exposed to the environment.
  - Exploitation: Utilize knowledge

Agent types

- Utility-based agents:
  - Learn a utility function of states uses it to select actions to maximize the expected outcome utility.
- Q-learning agents:
  - Learns the expected utility of taking a particular action a in a particular state s (Q-value of the pair (s, a)).
- Reflex agents:
  - Learn a policy that maps directly from states to actions

Passive Learning

Assumptions

- The agent's policy is fixed – in a state s it always executes the action $a = \pi(s)$
- Transition model $P(s'|s, a)$ is not known.
- Reward function for **each** state $R(s)$ is unknown

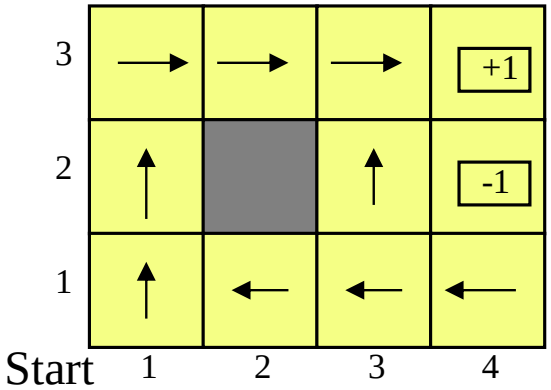
State-based representation in a fully observable environment

- Goal: Learn the utilities function $U^{\pi}(s)$ (or action-state pairs)

State-based representation in a fully observable environment

- Goal: Learn the utilities function $U^{\pi}(s)$ (or action-state pairs)
- Example:

Optimal policy for a 4x3 example,
with $\gamma = 1$ and $R(s) = -0.04$



The utilities of states

0.812	0.868	0.912	+1
0.762		0.660	-1
0.705	0.655	0.611	0.388
1	2	3	4

Passive learning task problem

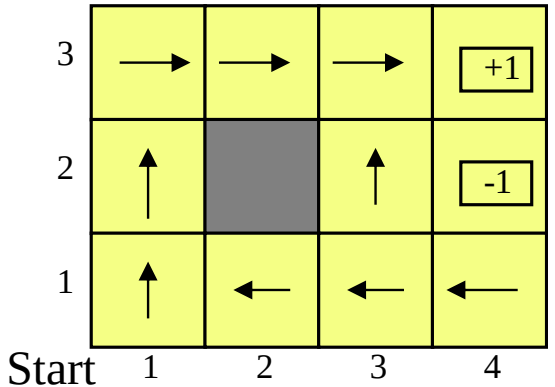
- The passive learning task is equivalent to the **policy evaluation** task of the **policy iteration** algorithm
- The main difference is that the passive learning agent does not know the **transition model** $P(s'|s,a)$
- Reward function for **each** state $R(s)$ is unknown

Solution?

Solution to passive learning task

- Step 1:
  - The agent executes a set of **trials** in the environment using its policy π

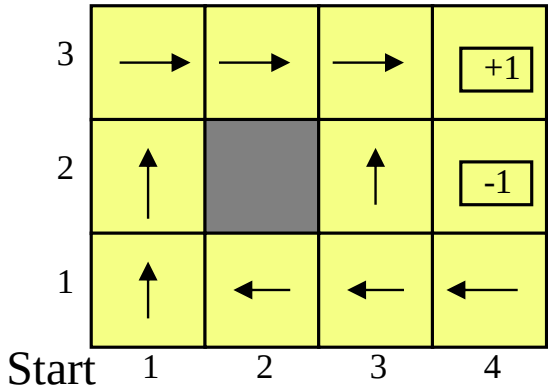
Optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$



Solution to passive learning task

- Step 1:
  - The agent executes a set of **trials** in the environment using its policy π
- Step 2
  - In each trial, the agent starts in state (1,1) and experiences a sequence of state transitions until it reaches one of the terminal states, (4,2) or (4,3).
  - Its percepts supply both the current state and the reward received in that state.

Optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$

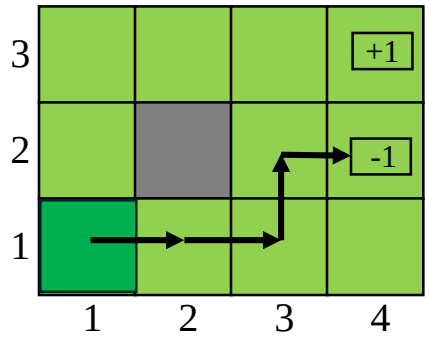


Example of different trials

Trial 1:

$(1,1)_{-0.04} \rightsquigarrow (2,1)_{-0.04} \rightsquigarrow (3,1)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (4,2)_{-1}$

Here, each percept of a state is associated with the reward received

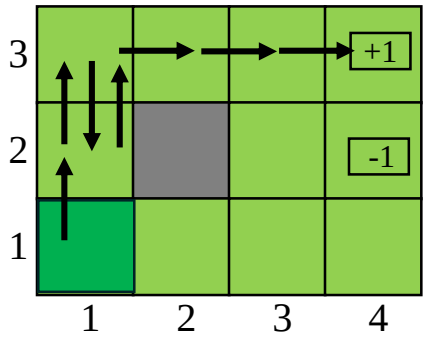


Example of different trials

Trial 2:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

Here, each percept of a state is associated with the reward received

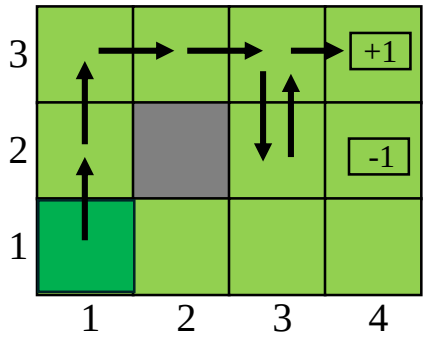


Example of different trials

Trial 3:

$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$

Here, each percept of a state is associated with the reward received



Utility computation

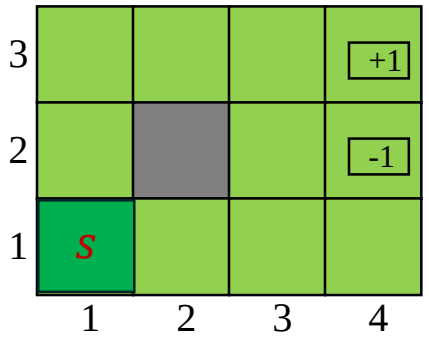
- The goal is to use the information about rewards to learn the utility $U^{\pi}(s)$ associated with each nonterminal state s given by the expected sum of discounted rewards obtained if policy π is executed:

$$U^{\pi}(s = s_0) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]$$

Direct utility estimation

- The main idea is that the utility $U^{\pi}(s)$ of a state s is the expected total reward from s onward (called the expected **reward-to-go**), while each ***trial*** sequence provides a *sample* of this quantity for each state visited by policy π!

- Let's see our examples for $\gamma = 1$:



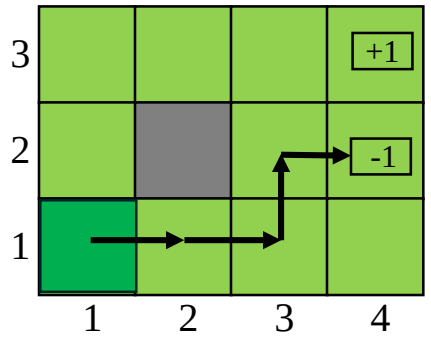
Example

- For the **first** trial:

$$(1,1)_{-0.04} \rightsquigarrow (2,1)_{-0.04} \rightsquigarrow (3,1)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (4,2)_{-1}$$

the expected utility is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 4 \times (-0.04) + (-1) = -1.16$$



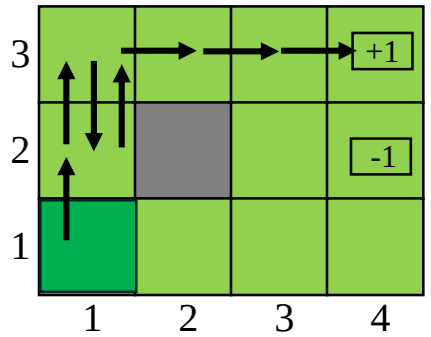
Example

- For the **second** trial:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility for $s = (1,1)$ is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 7 \times (-0.04) + 1 = 0.72$$



Example

- For the **second** trial:

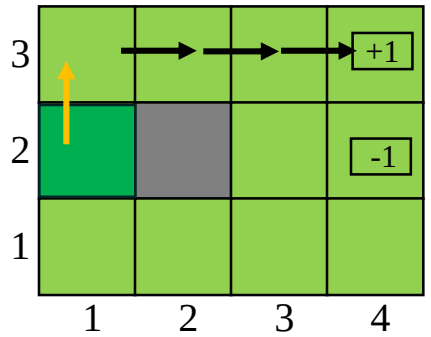
$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility for $s = (1,1)$ is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 7 \times (-0.04) + 1 = 0.72$$

- For $s = (1,2)$ we have two values:

$$U^{\pi}(1,2) = 4 \times (-0.04) + 1 = 0.84$$



Example

- For the **second** trial:

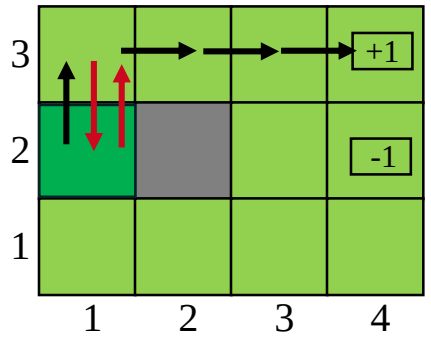
$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility for $s = (1,1)$ is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 7 \times (-0.04) + 1 = 0.72$$

- For $s = (1,2)$ we have two values:

$$U^{\pi}(1,2) = 4 \times (-0.04) + 1 = 0.84$$
$$U^{\pi}(1,2) = 6 \times (-0.04) + 1 = 0.76$$



Example

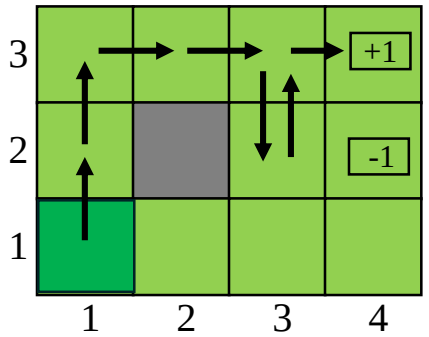
- For the **third** trial:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 4 \times (-0.04) + (-1) = 0.68$$

Remark: this policy provided lower utility in the first sequence!



Example

- For the **third** trial:

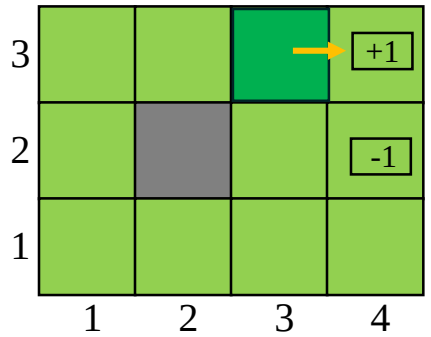
$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 4 \times (-0.04) + (-1) = 0.68$$

- For $s = (3,3)$ we have two values:

$$U^{\pi}(3,3) = 1 \times (-0.04) + 1 = 0.96$$



Example

- For the **third** trial:

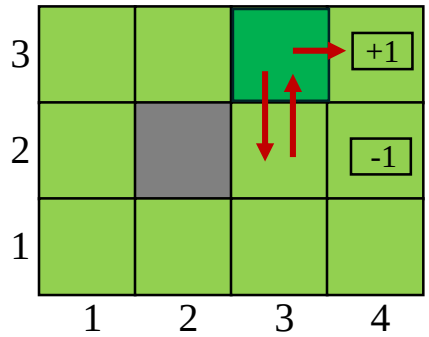
$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

the expected utility is given by

$$U^{\pi}(1,1) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{\gamma=1} = 4 \times (-0.04) + (-1) = 0.68$$

- For $s = (3,3)$ we have two values:

$$U^{\pi}(3,3) = 1 \times (-0.04) + 1 = 0.96$$
$$U^{\pi}(3,3) = 3 \times (-0.04) + 1 = 0.88$$



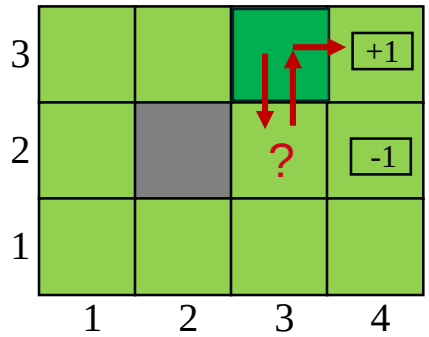
Dependence between the states

- For $s = (3,3)$ we have two values:

$$U^{\pi}(3,3) = 1 \times (-0.04) + 1 = 0.96$$
$$U^{\pi}(3,3) = 3 \times (-0.04) + 1 = 0.88$$

- What is the utility of $s = (3,2)$?
  - *Hint*: use Bellman equation!

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) U^{\pi}(s')$$

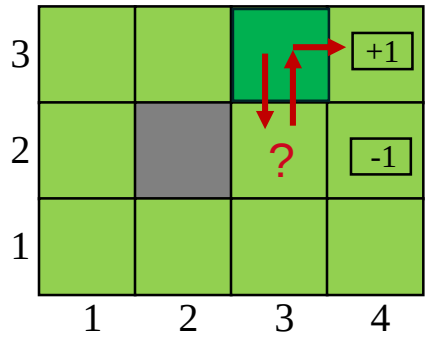


Remark

- Remember that the utilities of states are not independent!

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) U^{\pi}(s')$$

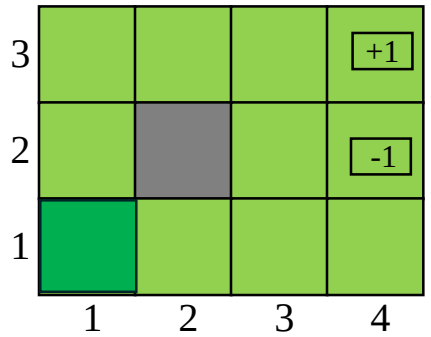
Logic: Since state (3,2) leads to a high utility state (3,3) it is also likely to have high utility!



Direct utility estimation algorithm

- At the end of each sequence, we compute the following
  - Observed **reward-to-go** for each state
  - Update the estimated utility for that state accordingly by a running average for each state in a table.
- In the limit of infinitely many trials, the sample average will converge to the true expectation

$$E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]$$



Remark

- Direct utility estimation is just a sample of supervised learning
  - Each example has the **state** as input and the observed **reward-to-go** as output.

Adaptive dynamic programming

ADP agent

- For a passive learning agent, an **adaptive dynamic programming** (or ADP) agent solves the corresponding Markov decision process using a dynamic programming method!

ADP agent

- For a passive learning agent, an **adaptive dynamic programming** (or ADP) agent solves the corresponding Markov decision process using a dynamic programming method!
- ADP agents computes the utilities of the states by plugging the learned transition model $P(s'|s,\pi(s))$ and the observed rewards $R(s)$ into the Bellman equations
- The utilities it needs to learn are those defined by
- the *optimal* policy;

linear (no max operator)

ADP agent

- For a passive learning agent, an **adaptive dynamic programming** (or ADP) agent solves the corresponding Markov decision process using a dynamic programming method!
- ADP agents computes the utilities of the states by plugging the learned transition model $P(s'|s,\pi(s))$ and the observed rewards $R(s)$ into the Bellman equations

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s'|s,\pi(s))U^{\pi}(s')$$

linear (no max operator)

- **Solution**: see the Policy iteration (**modified policy iteration)** algorithm in the previous lecture!!!

$$U_{i+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s'|s,\pi_i(s))U_i(s')$$

Learning principle

- The process of learning the model is easy because the environment is fully observable
  - This means that we have a supervised learning task where the input is a state–action pair and the output is the resulting state
  - Here, the transition model is represented by a table of probabilities $P(s'|s, \pi(s))$
- **Goal**:
  - Keep a track of how often each action outcome occurs and estimate the transition probability $P(s'|s, a = \pi(s))$ from the frequency with which s' is reached when executing $a = \pi(s)$ in s

Example

- For the trial sequences:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

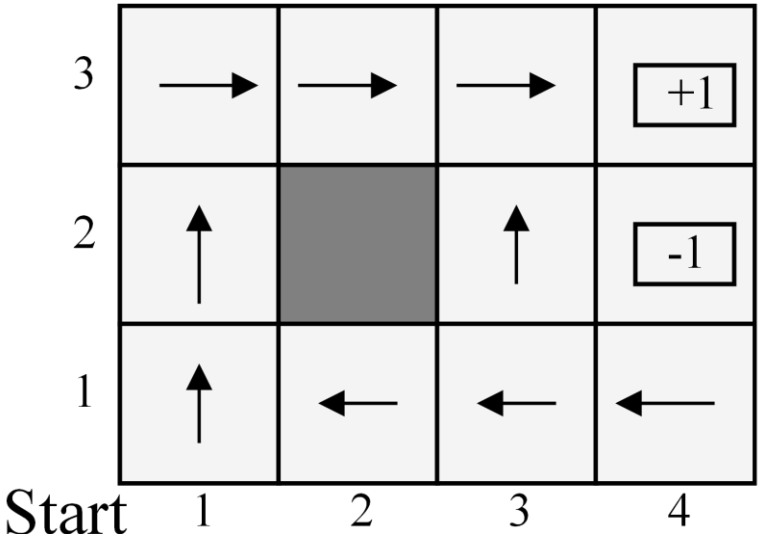
$$(1,1)_{-0.04} \rightsquigarrow (2,1)_{-0.04} \rightsquigarrow (3,1)_{-0.04} \rightsquigarrow (3,2)_{-0.04} \rightsquigarrow (4,2)_{-1}$$

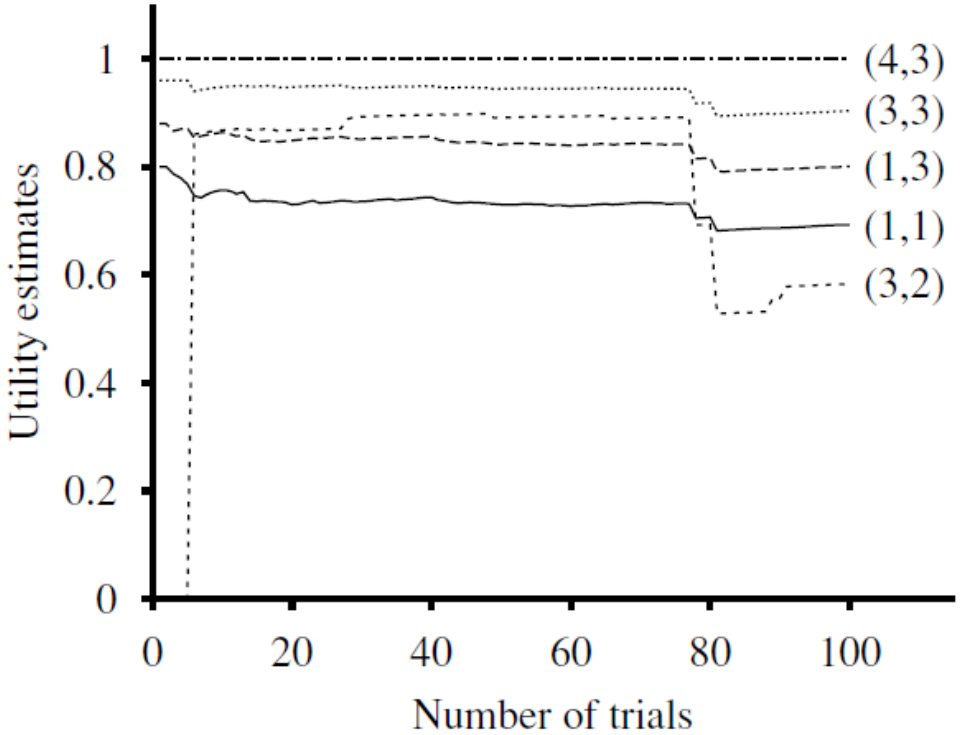
- The action "$a = Right$" is executed three times in $s = (1,3)$ and two out of three times the resulting state is $s' = (2,3)$

$$P((2,3)|(1,3), a = Right) = \frac{2}{3}$$

The passive ADP learning curves for the 4×3 world

Optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$





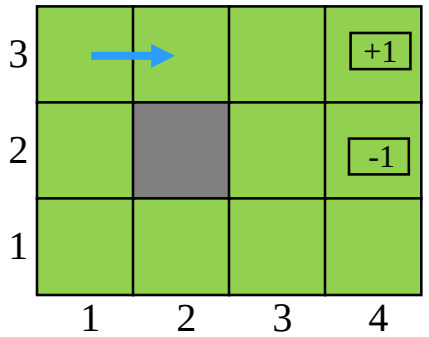
Remarks

- ADP agent is limited by its ability to learn the transition model
- ADP is a standard for comparison of other reinforcement learning algorithms
- However, ADP is intractable for large state spaces.

Temporal-difference learning

Use observed transitions to adjust the utilities

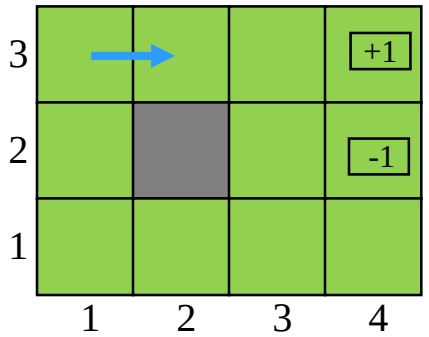
- TD: Adjust the estimated utility value of the current state based on its immediate reward and the estimated value of the next state.



Use observed transitions to adjust the utilities

- TD: Adjust the estimated utility value of the current state based on its immediate reward and the estimated value of the next state.
- Let's check the third trial sequence:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

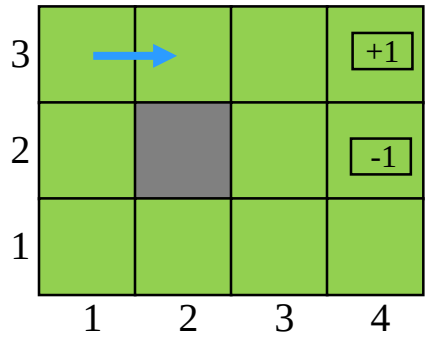


Use observed transitions to adjust the utilities

- TD: Adjust the estimated utility value of the current state based on its immediate reward and the estimated value of the next state.
- Let's check the third trial sequence:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

- Assume in the second trial we estimated $U^{\pi}(1,3) = 0.84$ and $U^{\pi}(2,3) = 0.92$



Use observed transitions to adjust the utilities

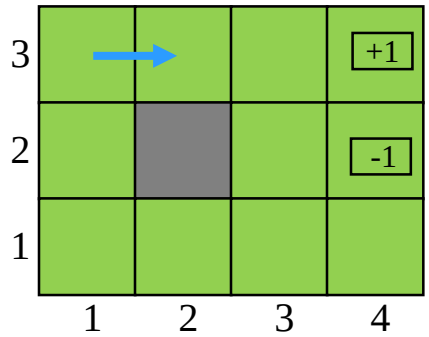
- TD: Adjust the estimated utility value of the current state based on its immediate reward and the estimated value of the next state.
- Let's check the third trial sequence:

$$(1,1)_{-0.04} \mapsto (1,2)_{-0.04} \mapsto (1,3)_{-0.04} \mapsto (2,3)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (3,2)_{-0.04} \mapsto (3,3)_{-0.04} \mapsto (4,3)_{+1}$$

- Assume in the second trial we estimated $U^{\pi}(1,3) = 0.84$ and $U^{\pi}(2,3) = 0.92$
- Thus, for transition in blue we estimate the utility:

$$U^{\pi}(1,3) = -0.04 + U^{\pi}(2,3) = 0.88$$

Hence, the current estimate of $U^{\pi}(1,3) = 0.84$ is low and should be updated



Temporal difference method

- TD: Adjust the estimated utility value of the current state based on its immediate reward and the estimated value of the next state.

- When a transition occurs from state s' to state s, the updating rule is given by

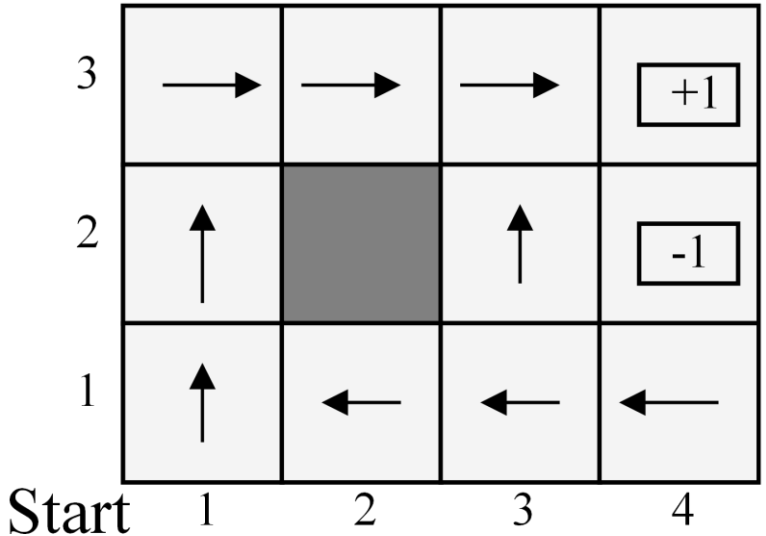
$$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha[R(s) + \gamma U^{\pi}(s') - U^{\pi}(s)]$$

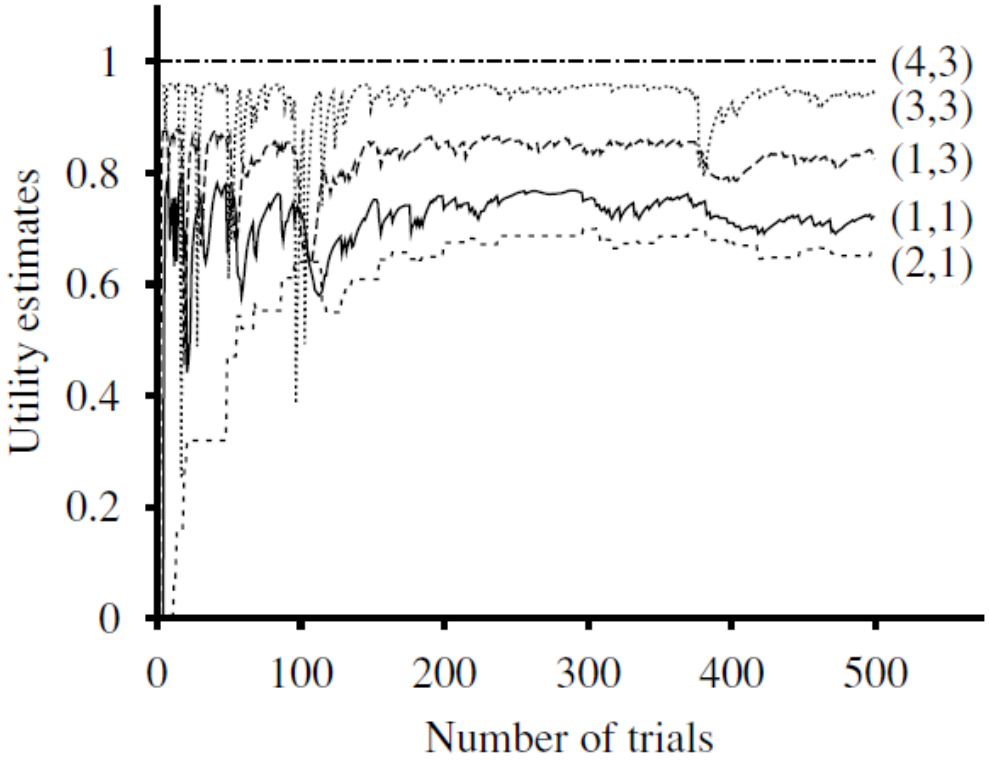
Difference in utilities between successive states; temporal difference

  - α is the learning rate parameter

The TD learning curves for the 4 × 3 world

Optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$





Active Learning

Notes

- A passive learning agent has a fixed policy π that determines its behavior
- An active agent must ***decide*** what ***actions*** to take (from *experiences*)
  - What their outcomes may be (both on learning and receiving the rewards in the long run)?

Notes

- A passive learning agent has a fixed policy π that determines its behavior
- An active agent must ***decide*** what ***actions*** to take (from *experiences*)
  - What their outcomes may be (both on learning and receiving the rewards in the long run)?

- Agent learns a complete model with outcome probabilities for all actions, rather than just the model for the fixed policy
  - Passive ADP agent, and
  - Considering that the agent has a choice of actions
- How to do it?

Active learning formulation

- The utilities it needs to learn are those defined by the *optimal* policy by Bellman equations:

$$U(s) = R(s) + \gamma \max_{\boldsymbol{a}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

Active learning formulation

- The utilities it needs to learn are those defined by the *optimal* policy by Bellman equations:

$$U(s) = R(s) + \gamma \max_{\boldsymbol{a}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

- For a given, action we compute $U(s)$ by using the **value** or **policy iteration** algorithms in the previous lecture

Active learning formulation

- The utilities it needs to learn are those defined by the *optimal* policy by Bellman equations:

$$U(s) = R(s) + \gamma \max_{\boldsymbol{a}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

- For a given, action we compute $U(s)$ by using the **value** or **policy iteration** algorithms in the previous lecture
  - By computing a utility function $U(s)$ that is optimal for the learned model, the agent extracts an optimal action by one-step look-ahead to maximize the expected utility

Active learning formulation

- The utilities it needs to learn are those defined by the *optimal* policy by Bellman equations:

$$U(s) = R(s) + \gamma \max_{\boldsymbol{a}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

- For a given, action we compute $U(s)$ by using the **value** or **policy iteration** algorithms in the previous lecture
  - By computing a utility function $U(s)$ that is optimal for the learned model, the agent extracts an optimal action by one-step look-ahead to maximize the expected utility
  - Alternatively, if it uses policy iteration, the optimal policy is already available, so it should simply execute the action the optimal policy recommends

Active learning formulation

- The utilities it needs to learn are those defined by the *optimal* policy by Bellman equations:

$$U(s) = R(s) + \gamma \max_{\boldsymbol{a}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

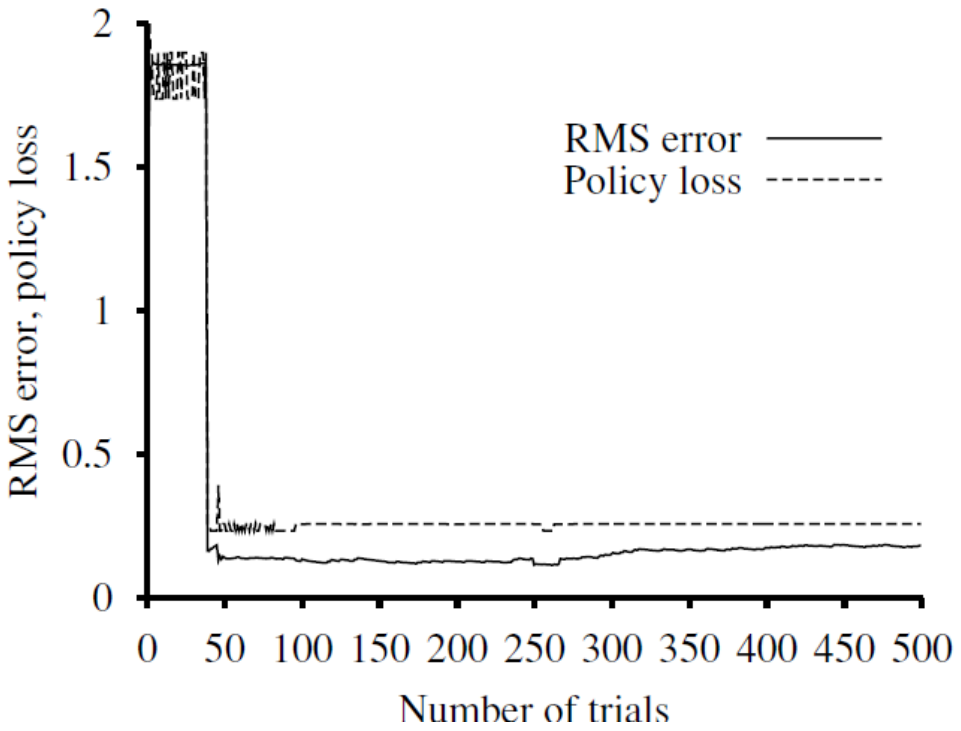
- For a given, action we compute $U(s)$ by using the **value** or **policy iteration** algorithms in the previous lecture
  - By computing a utility function $U(s)$ that is optimal for the learned model, the

But can the agent learn the true utilities or the true optimal policy!

should simply execute the action the optimal policy recommends

Greedy agent

- In the 39th trial, it finds a policy that reaches the +1 reward along the lower route via **(2,1), (3,1), (3,2), and (3,3)**
- After experimenting with minor variations, from the 276th trial onward, it sticks with this policy, never learning the utilities of the other states and never finding the optimal route via **(1,2), (1,3), and (2,3)**
  - **Greedy agent**.



Remarks

- How can it be that choosing the optimal action leads to suboptimal results?
  - The answer is that the learned model is not the same as the true environment; what is optimal in the learned model can therefore be suboptimal in the true environment
- Unfortunately, the agent does not know what the true environment is, so it cannot compute the optimal action for the true environment
  - What, then, is to be done?
- What the greedy agent has overlooked is that actions do more than provide rewards according to the current learned model
  - Actions also contribute to learning the **true model** by affecting the percepts that are received.

Exploitation vs Exploration

- By improving the model, the agent will receive greater rewards in the future
- An agent therefore must make a tradeoff between **exploitation** to maximize its reward—as reflected in its current utility estimates—and **exploration** to maximize its long-term well-being

- Since the learned model is not a copy of the real environment, the agent needs to trade off between exploration and exploitation:
  - Exploration: With a probability, ε, the agent selects the action randomly, maximizing the agent's long-term performance
  - Exploitation: With a probability, $1 - \varepsilon$, the agent selects the best known action, which is based on its policy.

Learning an action-utility function

Let’s construct an active temporal-difference learning agent

Note

- Now, the agent is no longer equipped with a fixed policy
  - To learn a utility function U, it will need to learn a model to be able to choose an action based on U via one-step look-ahead.
- Q-learning…

Q-learning principle

- **Q-learning**, which learns an action-utility representation $Q(s, a)$ instead of learning utilities
  - Model free learning
  - $Q(s, a)$ denotes the value of doing action *a* in state *s*

Q-learning principle

- **Q-learning**, which learns an action-utility representation $Q(s, a)$ instead of learning utilities
  - Model free learning
  - $Q(s, a)$ denotes the value of doing action *a* in state *s*
- Q-values are directly related to utility values as follows:

$$U(s) = \max_a Q(s, a)$$

Q-learning principle

- **Q-learning**, which learns an action-utility representation $Q(s, a)$ instead of learning utilities
  - Model free learning
  - $Q(s, a)$ denotes the value of doing action *a* in state *s*
- Q-values are directly related to utility values as follows:

$$U(s) = \max_a Q(s, a)$$

- Q-functions may seem like just another way of storing utility information, but they have a very important property

Model-free learning

- Given an estimated model, the update equation for Q-values is given by

$$Q(s,a) = R(s) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q(s',a')$$

- This does, however, require that a model also be learned, because the equation uses $P(s'|s,a)$

- Now, we introduce TD agent that learns a Q-function without the need of a model $P(s'|s,a)$, either for learning or for action selection

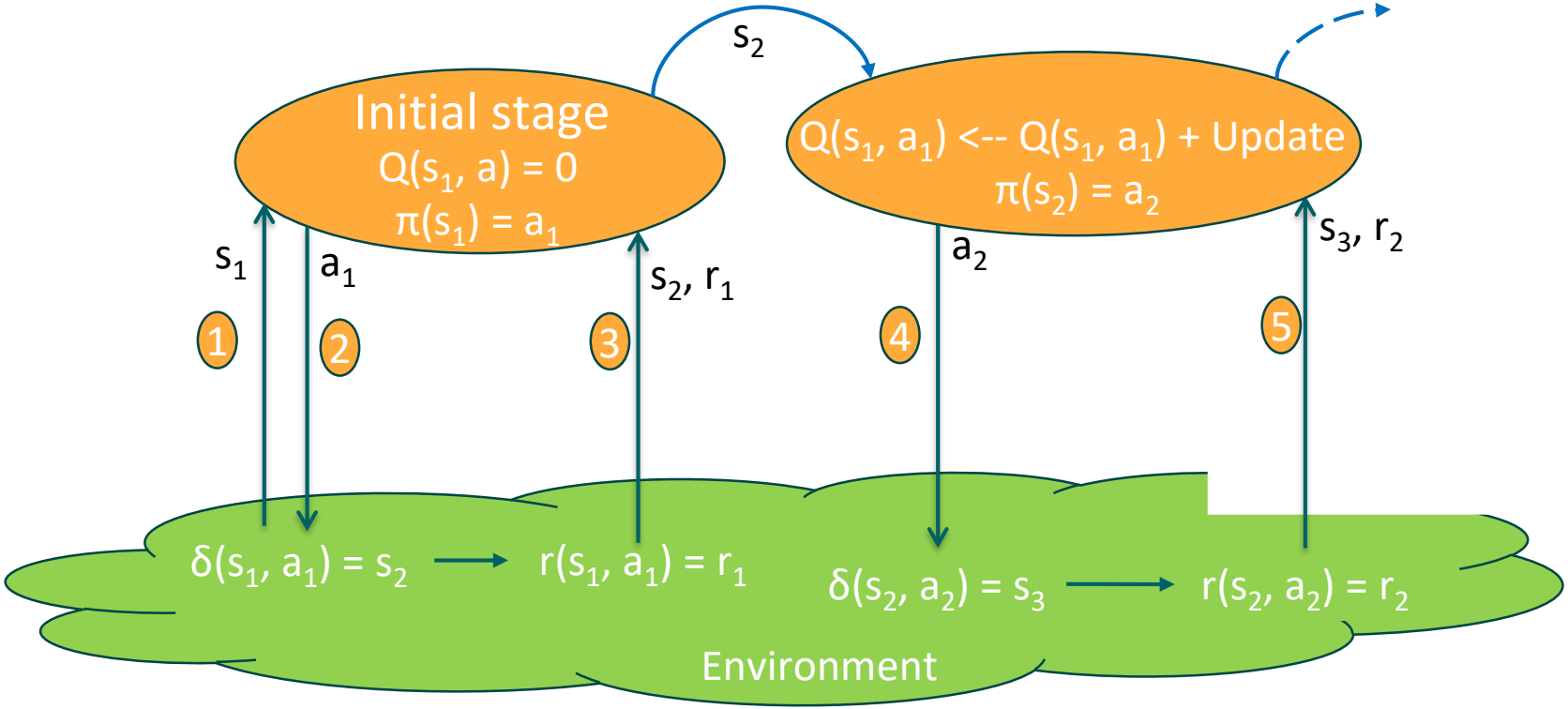
TD Q-learning

- The update equation for TD-Q learning is given by

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

  - $Q(s,a)$ is calculated whenever action a is executed in state s leading to state s'
  - α is the learning rate

Illustration of Q-value update process



State-Action-Reward-State-Action (SARSA)

- The update equation for TD-Q learning is given by

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s) + \gamma Q(s', a') - Q(s,a)]$$

  - Where a' is actual action taken in state s'

- The rule is applied at the end of each s, a, r, s', a' quintuplet—hence the name SARS

Q-learning algorithm principle

1. At *t*=0, initialize *Q*(s, a) for each pair (*s*, *a*)

```
// Adjust the estimated utility value of the current state based on its
immediate reward and the estimated value of the next state.
```

2. Observe the current state *s.*

3. Loop forever {

Select an action *a* with epsilon-greedy algorithm {Random with probability ϵ, and

$a = max_{a'} Q(s', a')$ with probability $1 - \epsilon$}

Receive immediate reward *r* and observe the new state s'.

Update $Q(s, a) \leftarrow Q(s, a) + \alpha[R(s) + \gamma \cdot max_{a'} Q(s', a') - Q(s, a)]$

Update s=s'

}

Remarks

Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$

SARSA: $Q(s,a) \leftarrow Q(s,a) + \alpha [R(s) + \gamma Q(s',a') - Q(s,a)]$

- Q-learning backs up the ***best*** Q-value from the state reached in the observed transition
- SARSA **waits until an action is actually taken** and backs up the Q-value for that action

- A greedy agent always takes the action with best Q-value and the two algorithms are identical.

Thank you!

Chair for Distributed Signal Processing

RWTH AACHEN UNIVERSITY